

SMART CAMPUS TASK MANAGER

Java CLI Project Report

VITyarthi – Build Your Own Project

Course Domain: Java Programming

Student Name: _____

Registration Number: _____

Section: _____

1. Introduction

Smart Campus Task Manager is a command-line Java application designed to help students maintain one local record of academic and personal tasks. It supports task creation, viewing, searching, sorting, completion, deletion, analytics, deadline reminders and CSV persistence.

2. Problem Statement

Students may maintain assignments, project deadlines and revision activities in scattered notes. This can make it harder to search tasks, identify pending work and retain information between program executions. The proposed system provides a lightweight, terminal-based task manager with persistent storage and deadline-oriented features.

3. Objectives

- Apply Java object-oriented programming in a real-world problem.
- Demonstrate collections using ArrayList and stream operations.
- Implement exception handling and input validation.
- Demonstrate file I/O through CSV persistence.
- Demonstrate multithreading through a deadline-reminder task.
- Build a modular project using user-defined packages.
- Provide a command-line interface that can be executed without a GUI.

4. Functional Requirements

FR1: Add, list, complete and delete tasks.

FR2: Search by task title or category.

FR3: Sort by due date.

FR4: Generate total/completed/pending/overdue/category analytics.

FR5: Run a separate reminder thread.

FR6: Save and reload tasks using CSV.

FR7: Validate empty fields, dates and task IDs.

5. Non-functional Requirements

Performance: Lightweight in-memory operations for student-sized data.

Usability: Numbered menu and clear prompts.

Reliability: Persistent CSV storage and handled load errors.

Maintainability: Separated packages and services.

Error Handling: Custom exceptions and validation.

Resource Efficiency: Buffered file I/O and a lightweight local data format.

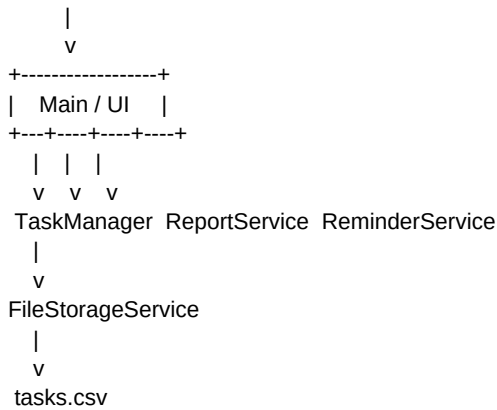
6. System Architecture

Main handles the CLI. TaskManager manages task state. FileStorageService handles CSV persistence. ReportService generates analytics. ReminderService implements Runnable for deadline checking. InputValidator centralizes validation.

7. Design Diagrams

System Architecture

```
+-----+
|  CLI User  |
+-----+-----+
```



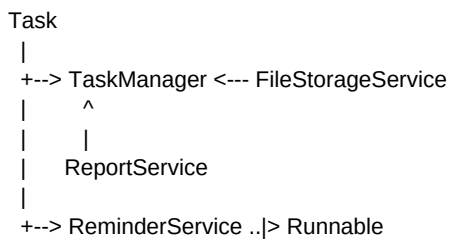
Workflow

Start -> Load CSV -> Menu
 -> Add -> Validate -> ArrayList -> Menu
 -> List/Search/Sort -> Display -> Menu
 -> Complete/Delete -> Validate ID -> Update -> Menu
 -> Analytics -> Report -> Menu
 -> Reminder -> Start Thread -> Check -> Menu
 -> Save & Exit -> Write CSV -> End

Use Case Diagram

Student --> Add Task
 Student --> View Tasks
 Student --> Complete Task
 Student --> Delete Task
 Student --> Search Task
 Student --> Sort by Due Date
 Student --> View Analytics
 Student --> Run Deadline Reminder
 Student --> Save & Exit

Class / Component Diagram



Main --> TaskManager
 Main --> FileStorageService
 Main --> ReportService
 Main --> ReminderService

Sequence Diagram

Student -> Main: Select Add Task
 Main -> InputValidator: Validate fields
 InputValidator --> Main: Valid
 Main -> TaskManager: addTask()
 TaskManager -> Task: create object
 Task --> TaskManager: Task
 TaskManager --> Main: Created task
 Main --> Student: Confirmation

Storage / ER Representation

TASK(id PK, title, category, dueDate, completed)

8. Design Decisions & Rationale

ArrayList: Appropriate for the intended small collection and sequential operations.

LocalDate: Reliable date parsing and comparison.

CSV: Simple, readable local persistence with no external dependency.

Runnable: Demonstrates multithreading.

Packages: Improve separation of concerns.

Custom exceptions: Make expected errors explicit.

9. Implementation Details

The project contains Main, Task, TaskManager, FileStorageService, ReportService, ReminderService, InputValidator, InvalidTaskException and TaskNotFoundException, plus TaskManagerTest. It uses encapsulation, Strings, ArrayList, streams/lambdas, java.io/java.nio.file, java.time, try/catch, custom exceptions, Runnable and Thread.

10. Screenshots / Results

Verified execution demonstrates task creation, listing, analytics and the threaded deadline reminder. The test suite verifies creation, completion, search, missing-task exception handling and invalid-input validation. A reproducible terminal transcript is included.

Representative verified terminal output:

```
=== SMART CAMPUS TASK MANAGER ===
```

1. Add Task
2. List Tasks
3. Complete Task
4. Delete Task
5. Search
6. Sort by Due Date
7. Analytics Report
8. Run Deadline Reminder (Thread)
9. Save & Exit

```
===== TASK ANALYTICS =====
```

```
Total tasks    : 4
Completed tasks : 1
Pending tasks   : 3
Overdue tasks   : 0
```

```
[Reminder Thread] Checking upcoming deadlines...
```

```
! Reminder: OS Assessment is due on 2026-09-20
```

```
[Reminder Thread] Check complete.
```

11. Testing Approach

TaskManagerTest.java uses Java assertions to verify adding tasks, completing a task, keyword search, missing-task exception handling and invalid-input validation. The project was compiled and the tests executed successfully with assertions enabled.

Test	Expected result	Status
Add task	Task count increases	PASS
Complete task	Completed count increases	PASS
Search	Matching task returned	PASS
Unknown ID	TaskNotFoundException	PASS
Empty title	InvalidTaskException	PASS

12. Challenges Faced

Challenges included coordinating CSV persistence with object state, validating dates, maintaining generated IDs after loading records, and handling InterruptedException while waiting for the reminder thread. These were addressed through dedicated services and explicit exception handling.

13. Learnings & Key Takeaways

The project provided practical experience with Java packages, classes, encapsulation, collections, streams, String processing, file I/O, custom exceptions, Runnable/Thread, modular design, CLI interaction and testing.

14. Future Enhancements

Possible extensions include user accounts, recurring tasks, priority levels, JSON/database storage, synchronized multi-threaded operations, configurable reminders, exportable reports, a REST API and a graphical/web interface.

15. References

1. Oracle Java Documentation – Java SE APIs.
2. Java Platform, Standard Edition Documentation – Collections Framework, I/O, Date-Time API and Concurrency.
3. VITyarthi Build Your Own Project instructions supplied for the course evaluation.